Assignment: Real-Time Seat Booking System

Objective

Design a scalable backend system for real-time seat booking with a focus on: Data consistency, Concurrency handling, Performance optimization, Clean architecture.

Problem Statement

Build a backend system that allows users to:

- View seat availability
- Select seats
- Temporarily reserve seats
- Confirm bookings

The system must handle **multiple users accessing and booking seats simultaneously**.

Core Functional Requirements

1. Seat Layout

- Provide seat layout for a specific event/show
- Each seat should include:
  - Unique identifier
  - Position (row/column or grid)
  - Current status (available / locked / booked)
- Support configurable layouts (different venues/events)

2. Seat Selection

- Users can select one or multiple seats
- Selected seats must be temporarily locked
- Locked seats should not be visible as available to other users

Constraints

1. Prevent Double Booking

- Ensure a seat cannot be booked by more than one user
- Must handle concurrent requests safely

2. Temporary Seat Lock

- Seat selection should create a temporary lock
- Lock duration should be configurable (e.g., 5 minutes)
- Locked seats must be inaccessible to other users

Business Rules

1. Auto Release of Seats

- Locked seats must automatically become available after timeout
- System should handle:
  - Expired locks
  - System failures or restarts

2. Booking Confirmation

- Users must confirm booking within lock duration
- On confirmation:
  - Seat status changes from locked to booked
- Booking process must be atomic (all or nothing)

Technical Expectations

1. Locking Strategy

- Implement a reliable mechanism to handle concurrent seat access
- Prevent race conditions and inconsistent data states
- Clearly define the approach used for locking

2. Performance Optimization

- Ensure system performs efficiently under high traffic
- Optimize:
  - Seat availability queries
  - Database operations
- Reduce unnecessary processing and latency

3. Data Integrity

- Maintain consistency across all operations
- Ensure no invalid seat states occur (e.g., booked without lock)

4. Scalability

- System should be designed to handle increasing users and requests
- Consider horizontal scaling and stateless design

Background Processing

- Implement automated handling for:
  - Expired seat locks
  - Cleanup of stale data
- Ensure background processes are reliable and fault-tolerant

Edge Cases to Consider

- Multiple users selecting the same seat simultaneously
- Lock expiration during booking flow